

# 编译器设计实验-第六次实验

计算机 2303 尤颢辰

2026 年 6 月 1 日

## 1 实验任务（必做）

### 1.1 任务描述

0. 整合前序实验的词法与语法/语义模块, 构成 Lab6 的基础;
1. 设计四元式数据结构与临时变量/标号生成;
2. 表达式翻译: 常量、变量、算术运算、数组访问、函数调用等;
3. 赋值语句翻译;
4. 控制流语句翻译, 条件跳转与回填;
5. 遍历 AST 驱动生成, 按序号输出完整四元式序列。

### 1.2 Task0: 前置任务与输入准备

Lab6 是编译器流水线的中间一环, 输入是 AST, 输出是四元式。为了让一条命令就能跑通整个逻辑, 我把前序实验的词法、语法、语义模块以函数库形式集成进来, 从而不产生任何中间文件, 三个阶段在内存里依次传递数据。

词法分析: 源自 Lab2 的 scanner.cpp, 把源程序切分为 token 序列;

语法 + 语义分析: 以 SLR(1) 分析表驱动归约, 建立抽象语法树 AST 并维护符号表, 同时完成类型检查; 其中 SLR(1) 分析表由 Lab4 离线生成, 作为只读资源载入。

因此这一步后我们得到了 AST 根节点与符号表, 后续的中间代码生成即在该 AST 上的一次后序遍历。

### 1.3 Task1: 设计数据结构, 生成临时变量/标号

中间代码生成需要解决三个最基本的问题:

1. 怎么表示一条指令? 我的方案是: 用四元组 (op, arg1, arg2, result) 来表示, 便于后续处理;
2. 子表达式的中间结果存哪? 比如  $a*b+c/d$  里  $a*b$  的结果没有名字。我的想法是: 创建一个临时变量分配器, 按需生成 T1、T2、T3 等中间结果。
3. 控制流跳转目标暂时不知道怎么办? 比如发 if 条件跳转时, 目标四元式还没生成。我的方案是: 需要 nextQuad 提供下一条编号、backpatch 在目标确定后回头填进去。

这三件事我封装进 IRegen 类, 让后续 Task 只需关心“翻译什么”, 不必反复维护这些底层细节。

```
1 struct Quad { string op, arg1, arg2, result; }; // 跳转类指令 result 存目标编号
2 class IRegen {
3     vector<Quad> code; // 四元式序列, 下标 i 对应编号 i+1
4     string newTemp(); // T1, T2, ...
5     int nextQuad() const; // 下一条四元式编号
6     int emit(op, a1, a2, res); // 追加一条, 返回其编号
```

```

7 void backpatch(quadNo, target); // 把第 quadNo 条的 result 填成目标编号
8 };

```

## 1.4 Task2: 表达式翻译

现在的问题是，AST 是树形结构，而四元式是线性指令序列，怎么把一棵表达式树翻译成正确的指令序列呢？课本的 place/code 翻译方案是一个很好的思路，每个表达式节点的翻译要同时做两件事：把计算过程要用到的四元式追加到公共序列里，以及返回这个表达式最终值的位置，（可能是常量字面量、变量名，也可能是新申请的临时变量）。递归翻译天然保证了“先算子表达式、再算父表达式”，运算优先级也由 AST 形状自动决定，到了翻译阶段就不用关心了。下面我们列出核心代码再举个例子分析：

```

1 case AST_BINOP: {
2     string l = genExpr(e->children[0]);
3     string r = genExpr(e->children[1]);
4     string t = newTemp();
5     emit(e->name, l, r, t); // op 取自节点名: +、-、*、/
6     return t;
7 }

```

我们就以  $c = 1 + 2 * 3$  中的  $+$  节点为例解释一下：

- 1.genExpr(children[0]): 递归翻译左子树。对  $+$  节点来说，左边是 Num(1)，直接返回字符串“1”；
- 2.genExpr(children[1]): 递归翻译右子树。右边是 BinOp( $*$ )，会先发射  $(*, 2, 3, T1)$ ，然后返回“T1”；两行执行完之后，四元式序列里已经有了  $(*, 2, 3, T1)$ ， $l = “1”$ ， $r = “T1”$ ；
- 3.newTemp() 一分配下一个临时变量，这里得到“T2”；
- 4.emit(e→name, l, r, t):e→name 就是节点存的运算符字符串“+”，所以发射  $(+, 1, T1, T2)$ ；
- 5.return t: 把“T2”返回给调用者，告诉上层“这个  $+$  表达式的值存在 T2 里”。

## 1.5 Task3: 赋值语句翻译

Task2 在 genExpr 中已经处理了数组读取 ( $=[]$ ) ——读取出现在表达式里，是值的来源；而写入 ( $[] =$ ) 出现在赋值语句的左侧，是值的去向，因此放到赋值分支里处理。

要实现这个功能，只需在增加一个 else if: 先用 genExpr 翻译右值得到 rhs，再翻译下标表达式得到 idx，发射 ( $[] =, rhs, idx, d$ ) 即可。核心代码如下：

```

1 else if (target->node_type == AST_ARRAY_ACCESS) {
2     string idx = genExpr(target->children[0]);
3     emit("[]=", rhs, idx, target->name); // ([]=, rhs, idx, d)
4 }

```

下面我们用一个简单的程序进行测试：

源程序：

```

1 int main() {
2     int a[10];
3     int b;
4     b = 3;

```

```

5   a[2] = b + 1;
6   b = a[2];
7   return 0
8 };
9 main()

```

```

PS E:\2026_Junior_S2\CompilerLab\lab6> .\lab6.exe demo_array.src slr1_table.txt

=== 语义检查 ===
无错误

=== 四元式 ===
1. (=, 3, , b)
2. (+, b, 1, T1)
3. ([]=, T1, 2, a)
4. ([]=, a, 2, T2)
5. (=, T2, , b)

```

图 1: Result

可以看到，目前我们的编译器已经可以处理所有的赋值语句翻译了。

## 1.6 Task4: 控制流语句翻译

控制流翻译的核心难点在于：写跳转指令时还不知道要跳到哪条，因为目标四元式尚未生成。解决办法是先发射一条目标为空的跳转，记录它的编号，等目标指令生成后再用 backpatch 把编号填进去。为此，我实现了一个辅助函数 genCondFalseJump，专门发射“条件为假时跳走”的四元式并返回其编号。

关于 if-else 的翻译，先调用 genCondFalseJump 发射一条 q\_false；接着翻译 then 分支；若有 else，再发射一条无条件跳 q\_skip 用于 then 结束后跳过 else，然后回填 q\_false 到当前位置，翻译 else 分支，最后回填 q\_skip 到 else 之后。

关于 while 的翻译，在翻译条件之前先记录 entry = nextQuad(); 调用 genCondFalseJump 发射 q\_false，翻译循环体，最后发射一条目标已知的无条件跳 (j, , , entry) 跳回条件入口，再回填 q\_false 到循环出口。

下面再验证一下输出如何：

源程序：

```

1  int main() {
2      int x;
3      int y;
4      x = 5;
5      if (x > 3) {
6          y = 1
7      } else {
8          y = 0
9      };
10     while (x > 0) {
11         x = x - 1
12     };

```

```

13     return 0
14 };
15 main()

```

```

PS E:\2026_Junior_S2\CompilerLab\lab6> .\lab6.exe demo_ctrl.src slr1_table.txt
=== 四元式 ===
1. (=, 5, , x)
2. (j<=, x, 3, 5)
3. (=, 1, , y)
4. (j, , , 6)
5. (=, 0, , y)
6. (j<=, x, 0, 10)
7. (-, x, 1, T1)
8. (=, T1, , x)
9. (j, , , 6)

```

图 2: Result

## 1.7 Task5: 遍历 AST，得到四元式序列

Task5 在 gen 中补全了剩余三类语句的翻译，使整个遍历覆盖文法中的所有语句类型。

输出语句：print E 先翻译表达式得到 place，发射 (print, place, , );

输入语句：input id 只需把目标变量名放进 result 位置，发射 (input, , , id);

return 语句：有返回值时翻译表达式后发射 (return, place, , ); 若返回值为空则发射 (return, , , )。

关于.src 文件末尾的 main()、calc() 等，其在 AST 中直接是 AST\_CALL 节点，挂在程序根节点下。

这样我们就实现了所有语句翻译为四元组的任务。

## 1.8 测试结果

选择代码库里比较完善的几个程序进行测试，先选择 23.src，其包含函数定义、形参、if-else 分支、函数调用与实参传递。结果：

```

PS E:\2026_Junior_S2\CompilerLab\lab6> .\lab6.exe 代码库\23.src slr1_table.txt
=== 四元式 ===
1. (j>=, a, b, 4)
2. (return, b, , )
3. (j, , , 5)
4. (return, a, , )
5. (param, 5, , )
6. (param, 8, , )
7. (call, max, 2, T1)
8. (=, T1, , result)
9. (return, 0, , )
10. (call, main, 0, T2)

```

图 3: Result

源代码:

```
1 int max(int a; int b;) {
2     if (a < b) {
3         return b
4     } else {
5         return a
6     };
7 };
8 int main() {
9     int result;
10    result = max(5, 8,);
11    return 0
12 };
13 main()
```

再测试 20.src, 其包含数组定义、数组元素读写和算术运算, 这是实现的一个难点。结果:

```
PS E:\2026_Junior_S2\CompilerLab\lab6> .\lab6.exe 代码库\20.src slr1_table.txt
=== 四元式 ===
1. ([]=, 1, 0, arr)
2. ([]=, 2, 1, arr)
3. (=[, arr, 0, T1)
4. (=[, arr, 1, T2)
5. (+, T1, T2, T3)
6. (return, T3, , )
7. (call, main, 0, T4)
```

图 4: Result

源代码:

```
1 int main() {
2     int arr[3];
3     arr[0] = 1;
4     arr[1] = 2;
5     return arr[0] + arr[1]
6 };
7 main()
```

再测试一个语义错误的 17.src, 结果:

```
PS E:\2026_Junior_S2\CompilerLab\lab6> .\lab6.exe 代码库\17.src slr1_table.txt
=== 语义检查 ===
共 2 条:
    未声明的标识符: c
    未声明的标识符: c
```

图 5: Result

可以看到对于错误情况，程序也能正常处理。

## 2 实验任务（选做）

### 2.1 任务描述 & 成果展示

本次选做任务主要是实现可视化平台的更新。目前已经完成了实验部分的所有可视化任务。展示如下：



图 6: 实验六部分



图 7: 实验七部分

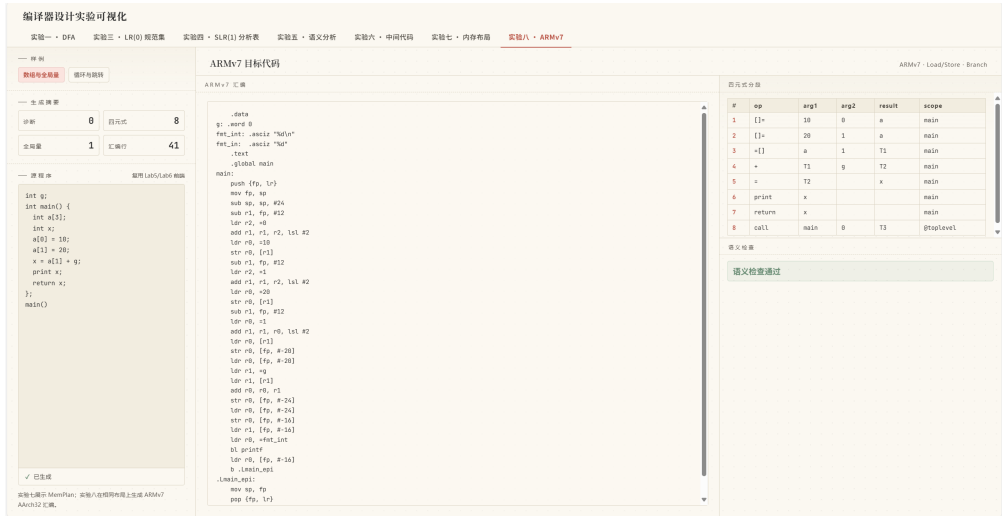


图 8: 实验八部分

已经发布到 github: <https://github.com/Yukino-chyan/CompilerWeb>。

### 3 个人总结

本次实验在前五次实验的基础上完成了编译器前端的最后一步——中间代码生成，整体集成在单一的.exe 文件里。实现过程中最有体会的是 place/code 翻译方案：每个表达式节点既要把计算过程的指令追加到公共序列、又要返回值的最终位置，理顺之后递归逻辑就非常自然了。另一个关键点是控制流的跳转回填——先发一条目标空着的跳转、记下编号，等目标确定后回头填进去。前五个 Lab 把源程序拆成结构化的 AST 和符号表，Lab6 则是重要的承上启下的一环：中间代码承接前端解析、衔接后端的内存分配与目标代码生成。经过这次实验，我对四元式作为前后端桥梁这一点有了更加直观的体会。